

JavaScript Output-Based Interview Questions

Part A: direct "what is X" concept questions (the ones interviewers ask before showing you code).

Part B: code → output → why, the classic trick questions.

Table of Contents

Part A — Core Concepts (Quick Definitions)

[A1](#) · [A2](#) · [A3](#) · [A4](#) · [A5](#) · [A6](#) · [A7](#) · [A8](#) · [A9](#) · [A10](#) · [A11](#) · [A12](#) · [A13](#) · [A14](#)

Part B — Output-Based Questions

[1](#) · [2](#) · [3](#) · [4](#) · [5](#) · [6](#) · [7](#) · [8](#) · [9](#) · [10](#) · [11](#) · [12](#) · [13](#) · [14](#) · [15](#) · [16](#) · [17](#) · [18](#) · [19](#) · [20](#) · [21](#) · [22](#) · [23](#) · [24](#) · [25](#) · [26](#) · [27](#) · [28](#) · [29](#) · [30](#)

Part A — Core Concepts (Quick Definitions)

A1. var vs let vs const

Answer: `var` is the old way — it can be redeclared, it belongs to the whole function (not just a block), and it starts as `undefined` even before you assign it. `let` and `const` only live inside the `{ }` block they're written in. `let` can be changed later, `const` can't be reassigned. We use `const` by default, `let` only when the value will change.

A2. Hoisting

Answer: Before running your code, JavaScript moves variable and function declarations to the top of their scope. `var` gets hoisted and set to `undefined`. Functions written with the `function` keyword are hoisted completely, so you can call them before they appear in the file. `let` / `const` are hoisted too, but stay unusable until their actual line.

```
console.log(a); // undefined
var a = 10;
```

A3. Temporal Dead Zone (TDZ)

Answer: This is the gap between the start of a block and the line where a `let` / `const` is actually declared. During this gap, using the variable throws an error

instead of giving `undefined` . It exists to stop us from using a variable before it's properly set up.

```
console.log(a); // ReferenceError
let a = 10;
```

A4. Closures

Answer: A closure is when a function remembers the variables from the place it was created, even after that outer function has already finished running.

```
function outer() {
  let count = 0;
  return () => ++count;
}
const add = outer();
add(); // 1
add(); // 2
```

Used constantly in React hooks, debounce functions, and anywhere we need to "remember" something between calls.

A5. `this` Keyword

Answer: `this` depends on **how** a function is called, not where it's written. In a regular function called like

`obj.method()`, `this` is `obj`. In an arrow function, `this` is taken from the surrounding code — arrow functions don't have their own `this`. That's why arrow functions are safer for callbacks.

A6. Call Stack

Answer: The call stack is where your normal JavaScript code actually runs, one function at a time, in order. When a function is called, it gets added (pushed) to the stack. When it finishes, it's removed (popped). JavaScript can only run one thing at a time because there's only one call stack.

A7. Event Loop

Answer: JavaScript is single-threaded, so it can only do one thing at a time on the call stack. Things like API calls or timers are handed off to run in the background, and once they're done, their callback goes into a queue. The event loop's job is simple: keep checking "is the call stack empty?" — if yes, take the next thing from the queue and run it.

A8. Microtask Queue vs Macrotask Queue

Answer: Promise callbacks (`.then` , `async/await`) go into the **microtask queue**. Things like `setTimeout` and `setInterval` go into the **macrotask queue**. The event loop always finishes **everything** in the microtask queue first, before it picks even one task from the macrotask queue. That's why a Promise always runs before a `setTimeout` , even if the timeout is 0ms.

A9. Promises

Answer: A Promise represents a value that will be available later, like the result of an API call. It has 3 states: **pending** (still waiting), **fulfilled** (success), and **rejected** (failed). Once it settles into fulfilled or rejected, it never changes again.

```
fetch("/api").then(res =>
  res.json()).catch(err => console.log(err));
```

A10. Async/Await

Answer: A cleaner way to work with Promises, so asynchronous code reads like normal step-by-step code instead of chaining `.then()` . An `async` function always returns a Promise. `await` pauses the

function until that Promise finishes, without blocking the rest of the app.

```
async function getUser() {
  try {
    const res = await fetch("/user/1");
    return await res.json();
  } catch (e) {
    console.log(e);
  }
}
```

A11. Primitive vs Reference Types

Answer: Primitive types (`string` , `number` , `boolean` , `null` , `undefined`) are copied **by value** — copying gives you a fully separate value. Reference types (`object` , `array` , `function`) are copied **by reference** — both variables point to the same thing in memory, so changing one changes the other.

A12. Shallow Copy vs Deep Copy

Answer: A shallow copy only copies the first level — if there's a nested object inside, the copy and the original still share that same nested object. A deep copy copies everything, including nested data, so the copy is completely independent. This matters a lot

when updating state in React/Redux — a shallow copy of nested data can cause bugs where the UI doesn't update.

```
const shallow = { ...original };
shallow.address.city = "X"; // also changes
original.address.city
```

A13. Debouncing

Answer: Debouncing means waiting until the user **stops** doing something for a set amount of time, and only then running the function — used on things like a search box, so we don't call the API on every single keystroke.

```
function debounce(fn, delay) {
  let timer;
  return (...args) => {
    clearTimeout(timer);
    timer = setTimeout(() => fn(...args),
      delay);
  };
}
```

A14. Currying

Answer: Currying means breaking a function that takes multiple arguments into a chain of functions that each take **one argument at a time**.

```
const multiply = (a) => (b) => (c) => a * b * c;  
multiply(2)(3)(4); // 24
```

Part B — Output-Based Questions

1. var with setTimeout

```
for (var i = 1; i <= 3; i++) {  
  setTimeout(() => console.log(i), 1000);  
}
```

Output: 4 4 4

Explanation: `var` is function-scoped, so there's only **one** `i` shared by all three callbacks. The loop finishes completely (`i` becomes 4) before any callback runs.

2. let with setTimeout

```
for (let i = 1; i <= 3; i++) {  
  setTimeout(() => console.log(i), 1000);  
}
```



```
}
```

Output: 1 2 3

Explanation: `let` is block-scoped, so each loop run gets its **own fresh copy** of `i`.

3. Promise vs `setTimeout`

```
console.log("Start");
setTimeout(() => console.log("Timeout"),
0);
Promise.resolve().then(() =>
console.log("Promise"));
console.log("End");
```

Output: Start End Promise Timeout

Explanation: Synchronous code runs first. Promise callbacks go to the Microtask Queue, `setTimeout` goes to the Macrotask Queue. The Microtask Queue is always cleared before the next Macrotask runs.

Interview Answer: "JavaScript is single-threaded and uses an Event Loop. Sync code runs first on the Call Stack. Promise callbacks go to the Microtask Queue, `setTimeout` goes to the Macrotask Queue. Once the stack is empty, the Event Loop runs all Microtasks before even one Macrotask — so Promise always wins over `setTimeout`, even with a 0ms delay."

4. Nested setTimeout

```
console.log("A");
setTimeout(() => {
  console.log("B");
  setTimeout(() => console.log("C"), 0);
}, 0);
console.log("D");
```

Output: A D B C

Explanation: A and D run first (sync). The first timeout prints B, and only then schedules the second timeout, which prints C after that.

5. Array Reference

```
let a = [1, 2, 3];
let b = a;
b.push(4);
console.log(a);
```

Output: [1, 2, 3, 4]

Explanation: Arrays are reference types — a and b point to the same array in memory, so changing one changes the other.

6. typeof null

```
console.log(typeof null);
```

Output: "object"

Explanation: A long-standing JavaScript bug that was never fixed (for backward compatibility reasons).

`null` is not actually an object.

7. == vs ===

```
console.log(1 == "1");  
console.log(1 === "1");
```

Output: true false

Explanation: `==` converts types before comparing (type coercion). `===` checks both value and type with no conversion.

8. var Hoisting

```
console.log(a);  
var a = 10;
```

Output: undefined

Explanation: `var` declarations are hoisted to the top, but only the declaration — not the value. So it exists as `undefined` until the assignment line runs.

9. let Hoisting (TDZ)

```
console.log(a);  
let a = 10;
```

Output: `ReferenceError: Cannot access 'a' before initialization`

Explanation: `let` is hoisted too, but it stays in the "Temporal Dead Zone" — unusable — until the line where it's actually declared.

10. Object Reference

```
let obj1 = { name: "John" };  
let obj2 = obj1;  
obj2.name = "Peter";  
console.log(obj1.name);
```

Output: `"Peter"`

Explanation: Same idea as arrays — objects are reference types, so `obj1` and `obj2` point to the same object.

11. Primitive Values

```
let a = 10;  
let b = a;  
b = 20;  
console.log(a, b);
```

Output: 10 20

Explanation: Primitives are copied by value. Changing `b` doesn't affect `a` at all — unlike objects/arrays.

12. typeof NaN

```
console.log(typeof NaN);
```

Output: "number"

Explanation: `NaN` stands for "Not a Number," but ironically its type is still `number`.

13. null vs undefined

```
console.log(null == undefined);  
console.log(null === undefined);
```

Output: true false

Explanation: `==` treats them as equal because of type coercion rules built into JS. `===` checks type too, and they're different types.

14. Closure

```
function outer() {  
  let count = 0;  
  return function () {  
    count++;  
    console.log(count);  
  };  
}  
const increment = outer();  
increment();  
increment();  
increment();
```

Output: 1 2 3

Explanation: The inner function "remembers" the `count` variable from `outer`, even after `outer` has already finished running. That's a closure.

15. Async Function Return Value

```
async function test() {  
  return 10;  
}  
test().then(console.log);
```

Output: 10

Explanation: An `async` function always wraps its return value in a Promise automatically, so `.then()` can read it.

16. `[] == false`

```
console.log([] == false);
```

Output: true

Explanation: JS converts `[]` to `""` and then to `0`, and `false` also becomes `0` — so they end up equal. This is exactly why we avoid `==` for comparisons.

17. Floating Point Math (0.1 + 0.2)

```
console.log(0.1 + 0.2);  
console.log(0.1 + 0.2 === 0.3);
```

Output: 0.30000000000000004 and false

Explanation: Computers store decimal numbers in binary, and some decimals (like 0.1) can't be stored exactly — so tiny rounding errors happen. This isn't a JS-only bug, it happens in most programming languages.

Interview Answer: "JavaScript uses floating-point math (IEEE 754), and some decimal numbers can't be represented exactly in binary, causing small rounding errors. To safely compare decimals, we check if the difference is smaller than a tiny number, like `Math.abs(a - b) < Number.EPSILON`."

18. String + Number Coercion in `+` and `-`

```
console.log("5" + 3);  
console.log("5" - 3);
```

Output: `"53"` and `2`

Explanation: `+` with a string triggers string concatenation, so `3` gets converted to text. `-` only works with numbers, so `"5"` gets converted to a number instead.

19. Default Array Sort


```
console.log([10, 1, 2].sort());
```

Output: [1, 10, 2]

Explanation: By default, `.sort()` converts everything to strings and sorts alphabetically, not numerically – so "10" comes before "2". To sort numbers correctly, pass a compare function: `.sort((a, b) => a - b)`.

20. `this` Losing Binding When Passed as a Callback

```
const user = {  
  name: "Asha",  
  greet() { console.log(this.name); },  
};  
const greetFn = user.greet;  
greetFn();
```

Output: undefined

Explanation: When you pull `greet` out and call it on its own, it loses its connection to `user`. `this` no longer points to `user`. Fix: use `.bind(user)`, or call it as `user.greet()`.

21. `this` in Regular Function vs Arrow Function as Object Method

```
const obj = {  
  name: "Asha",  
  regular() { console.log(this.name); },  
  arrow: () => console.log(this.name),  
};  
obj.regular();  
obj.arrow();
```

Output: "Asha" then undefined

Explanation: `regular()` gets `this` from how it's called (`obj.regular()` → `this` is `obj`). The arrow function doesn't have its own `this` — it uses `this` from the surrounding scope, which is not `obj`.

22. Optional Chaining on a Missing Property

```
const user = { profile: null };  
console.log(user.profile?.address?.city);
```

Output: undefined

Explanation: `?.` stops and safely returns `undefined` as soon as it hits something `null` / `undefined`, instead of crashing with "Cannot read property of null."

23. Promise.all with One Rejected Promise

```
Promise.all([
  Promise.resolve(1),
  Promise.reject("Error!"),
  Promise.resolve(3),
])
  .then((res) => console.log(res))
  .catch((err) => console.log(err));
```

Output: "Error!"

Explanation: `Promise.all` fails immediately if even one promise rejects — it doesn't wait for the others. If you need all results regardless of failure, use `Promise.allSettled` instead.

24. Falsy Coercion: `0 == "0"` vs `"" == "0"` vs `false == "0"`

```
console.log(0 == "0");
console.log("" == "0");
console.log(false == "0");
```

Output: true false true

Explanation: `0 == "0"` → the string `"0"` gets converted to the number `0`, so they match. `"" ==`

"0" → both sides are already strings, so `==` compares them directly as text with no conversion — and `""` is simply not the same text as `"0"`, so it's `false`. `false == "0"` → `false` becomes `0`, and `"0"` becomes `0`, so they match. This mix of "sometimes it converts, sometimes it doesn't" is exactly why `==` is confusing and `===` is preferred.

25. Closures Inside a Loop, Storing Functions in an Array

```
const funcs = [];  
for (var i = 0; i < 3; i++) {  
  funcs.push(() => console.log(i));  
}  
funcs[0]();  
funcs[1]();  
funcs[2]();
```

Output: 3 3 3

Explanation: Same root cause as Q1 — all three functions close over the same shared `i` (because of `var`), and by the time they run, the loop has already finished with `i = 3`. Switching to `let` would print `0 1 2`.

26. Async Function Without Await – Ordering

```
console.log("1");
async function test() {
  console.log("2");
  await null;
  console.log("3");
}
test();
console.log("4");
```

Output: 1 2 4 3

Explanation: Code in an `async` function runs synchronously **until** the first `await`. So "2" logs immediately. Once `await` is hit, the rest of the function ("3") is paused and moved to the microtask queue, so "4" (the remaining sync code outside) runs before it.

27. Spread Copy vs Reference (Array of Objects)

```
const original = [{ name: "Asha" }];
const copy = [...original];
copy[0].name = "Ravi";
console.log(original[0].name);
```

Output: "Ravi"

Explanation: Spread only does a **shallow copy** — it copies the array itself, but the objects inside are still the same references. Changing a nested object in the copy also changes it in the original.

28. Default Parameters Are Evaluated on Every Call

```
function addItem(item, list = []) {  
  list.push(item);  
  return list;  
}  
console.log(addItem(1));  
console.log(addItem(2));
```

Output: [1] then [2]

Explanation: A default parameter like `= []` creates a brand new array **every time the function is called** — it's not shared or reused between calls, so each call starts fresh.

29. String Immutability

```
let str = "hello";  
str[0] = "H";  
console.log(str);
```

Output: `"hello"`

Explanation: Strings in JavaScript are immutable — you cannot change a character by index. The assignment silently does nothing. To change it, you'd create a new string, e.g. `"H" + str.slice(1)`.

30. `Array.isArray` vs `typeof` for Arrays

```
console.log(typeof []);  
console.log(Array.isArray([]));
```

Output: `"object"` then `true`

Explanation: `typeof` can't tell an array apart from a regular object — both return `"object"`.

`Array.isArray()` is the correct, reliable way to check if something is actually an array.

[← Back to top](#)